Performance Test Plan

DemoBlaze E-Commerce Web Application

https://www.demoblaze.com

Apache JMeter 5.6.3 | Java 21 | March 2026

Introduction

This Performance Test Plan defines the approach, scope, tools, and schedule for conducting performance testing on the DemoBlaze web application. DemoBlaze is a publicly accessible e-commerce demonstration platform that simulates a real online store where users can browse products, add items to a shopping cart, and complete a purchase.

The purpose of this document is to ensure that all stakeholders have a clear and shared understanding of how performance testing will be conducted, what success looks like, and what risks have been identified. The testing is conducted using Apache JMeter 5.6.3 running on Java 21.

Test Objectives

The performance testing effort aims to achieve the following objectives.

- **Measure Response Time:** Determine how long it takes for the application to respond to each user request under varying levels of concurrent load. The target is that all critical transactions complete within 2,000 milliseconds on average.
- **Measure Throughput:** Determine how many requests the application can successfully process per second. The target is at least 500 requests per second under peak load conditions.
- **Measure Error Rate:** Determine the percentage of requests that result in failures under load. The target is that no more than 1 percent of all requests result in an error under any test phase.

- **Assess Scalability:** Evaluate how the application behaves as the number of concurrent users increases from 50 to 1,000. The application should degrade gradually and predictably rather than failing suddenly.
- **Identify Bottlenecks:** Discover which transactions or endpoints degrade or fail first under load so that the development team can target the right areas for optimisation.
- **Validate Endurance:** Confirm that the application can sustain acceptable performance over an extended period of continuous operation without memory leaks or resource exhaustion.

Scope of Testing

In Scope

This subsection outlines the performance testing activities and system behaviours that will be evaluated as part of this engagement to ensure the DemoBlaze application performs reliably under expected and extreme load conditions.

- **Load Testing:** Simulating expected concurrent user loads across core workflows including homepage navigation, user authentication, product detail page viewing, add-to-cart operations, and end-to-end checkout to verify acceptable performance at anticipated traffic levels.
- **Stress Testing:** Incrementally increasing concurrent users beyond expected peak loads to identify the system's maximum capacity and observe behaviour under overload, including error rates and failure modes.
- **Scalability Testing:** Evaluating how application performance metrics , response time, throughput, error rate , scale as the number of virtual users increases from 50 to 1,000, to confirm linear or graceful degradation rather than abrupt failure.
- **Endurance Testing:** Running sustained workloads over an extended period of 10 minutes to detect memory leaks, session handling issues, or gradual response time degradation under prolonged usage.
- **Transaction-Level Performance Monitoring:** Capturing and analysing response times, throughput, and error rates at the individual transaction level across all five in-scope user journey steps: Homepage, Login, View Product, Add to Cart, and Place Order.

- **Performance Baseline Establishment:** Documenting baseline performance metrics for all in-scope transactions under controlled load conditions of 50 concurrent users to serve as the reference point for all subsequent test phases.

Out of Scope

This subsection identifies testing activities and system components that are not included in the current performance testing engagement to ensure the scope remains focused on the defined objectives.

- **Functional Regression Testing:** This plan is concerned only with how fast and how reliably the application responds under load. Whether the application produces the correct business output is a separate concern handled by functional testing and is not evaluated here.
- **Mobile Application Performance:** This engagement covers the DemoBlaze web application only. Testing the mobile interface requires different tools and a different approach and falls outside the boundaries of this plan.
- **Database-Level Load Testing:** The test team has no direct access to the DemoBlaze database layer. Direct database stress testing and query plan analysis are not possible and are excluded from this engagement.
- **Third-Party CDN and Asset Performance:** External fonts, scripts, and images served by third-party content delivery networks are not under the test team's control and are not included in the measured response times.
- **Server-Side Infrastructure Tuning:** The DemoBlaze application is publicly hosted and its server infrastructure is not accessible to the test team. Any activities involving server configuration changes or database query optimisation are therefore outside the scope of this engagement.

Testing Resources

Human Resources

The following roles are involved in this performance testing engagement.

- **QA Engineer:** Responsible for designing the test plan, configuring the JMeter test plan, executing all test phases, analysing results, and producing the final performance report.
- **Product Manager:** Responsible for defining and formally approving the performance acceptance criteria and SLA thresholds before testing begins. Reviews test findings at the sprint review, provides business context for interpreting results, and gives final sign-off on test completion.
- **QA Lead:** Responsible for overseeing the entire performance testing effort, reviewing and approving the test plan before execution, monitoring test progress across all phases, validating findings, and ensuring all deliverables meet the required standard before submission.
- **Development Team:** Available to review performance findings, clarify API behaviour, and implement optimisation recommendations based on bottleneck analysis.

Testing Tools

- **Apache JMeter 5.6.3:** The primary load testing tool used to simulate concurrent virtual users, send HTTP requests, extract dynamic data, apply assertions, and generate performance reports.
- **Java 21 (Temurin LTS):** The Java runtime required to run Apache JMeter. JMeter 5.6.3 is fully compatible with Java 21.
- **GitHub Actions:** The CI/CD platform used to automate performance test execution, publish HTML Dashboard reports to GitHub Pages, and send email notifications with test results.

Performance Test Design Approach

This section describes the end-to-end approach taken to design, configure, execute, and report on performance tests for the DemoBlaze application.

I. Test Planning and Design

Define Test Scenarios

A test scenario is a defined sequence of HTTP requests that models a realistic user journey through the application. All virtual users in this test plan execute the same five-transaction sequence in strict order: Homepage, Login, View Product, Add to Cart, and Place Order. The order is enforced because later transactions depend on data produced by earlier ones. Specifically, the Login transaction must execute first because it produces the session token required by Add to Cart and Place Order.

Define Performance Metrics

The following performance metrics are collected and evaluated against defined SLA targets for each test phase.

- **Average Response Time:** The mean elapsed time across all samples for a given transaction. Target: below 2,000 milliseconds for Baseline, Load, Peak Load, and Endurance phases. During the Stress Test this threshold is expected to be breached as the application is pushed beyond its capacity.
- **90th Percentile Response Time (P90):** The response time below which 90 percent of all requests complete. Target: below 3,000 milliseconds during Load and Peak Load phases. During the Stress Test, P90 values are recorded and documented as evidence of degradation rather than measured against a pass threshold.
- **95th Percentile Response Time (P95):** The response time below which 95 percent of all requests complete. Monitored across all phases including Stress and Endurance to identify the worst-case experience for users at the slow end of the distribution.
- **Throughput:** The number of requests processed per second. Target: at least 500 requests per second at Peak Load. During the Stress Test, throughput decline below

this threshold is expected and recorded as the breaking point indicator. During the Endurance Test, throughput is monitored for stability over the full 10-minute run.

- **Error Rate:** The percentage of samples that fail any assertion. Target: below 1 percent for Baseline, Load, Peak Load, and Endurance phases. During the Stress Test, error rate is expected to exceed this threshold and is recorded to document the failure behaviour of the application under extreme load.
- **Response Time Stability (Endurance):** Tracked exclusively during the Endurance Test. Response times and error rates are monitored across the full 10-minute run for any upward trend. A stable flat line indicates no memory leaks or resource exhaustion. A rising trend indicates a degradation problem that requires investigation.
- **Latency:** The time to first byte received from the server, reflecting network and initial server processing time. Recorded across all phases including Stress and Endurance.

II. JMeter Test Setup

HTTP Request Samplers

An HTTP Request Sampler in JMeter represents a single HTTP request sent to the server. Each of the five transactions in this test plan is implemented as a dedicated HTTP Request Sampler. The samplers are configured with the correct HTTP method, host, port, path, and request body for each endpoint. All API samplers use HTTPS on port 443 targeting api.demoblaze.com, while the Homepage and View Product samplers target www.demoblaze.com.

Thread Groups

A Thread Group in JMeter represents a group of simulated concurrent users. Each thread in the group executes the full five-transaction sequence independently. This test plan contains five thread groups, each corresponding to a distinct test phase. Only one thread group is enabled at a time during test execution.

- **Baseline:** 50 concurrent users, 50-second ramp-up, 1 loop per user. Establishes the reference performance metrics under minimal load.

- **Medium Load Test:** 150 concurrent users, 100-second ramp-up, 2 loop per user. Validates performance under typical busy-day traffic.
- **Peak Load:** 500 concurrent users, 100-second ramp-up, 2 loop per user. Tests the application at maximum expected operational capacity.
- **Stress Test:** 1,000 concurrent users, 120-second ramp-up, 3 loops per user. Intentionally exceeds capacity to identify the breaking point.
- **Endurance Test:** 50 concurrent users, 100-second ramp-up, continuous loops for 600 seconds (10 minutes). Tests for memory leaks and long-term stability.

Timers

A Constant Timer is configured with a delay of 1,000 milliseconds between each consecutive request within every thread. This simulates think time, which is the time a real user spends reading a page or deciding what to click next before sending their next request. Without think time, virtual users would send requests as fast as the network allows, producing unrealistically aggressive load that does not reflect real-world usage patterns.

Assertions

Assertions in JMeter validate that each server response meets defined expectations. A sample is only counted as a pass when all assertions attached to its sampler succeed. The following assertions are applied to each sampler in this test plan.

- **HTTP 200 Response Code Assertion:** Applied to all five samplers. Fails the sample if the HTTP status code returned by the server is anything other than 200 OK. This catches server errors such as 401 Unauthorised, 404 Not Found, and 500 Internal Server Error.
- **Response Body Assertion:** Applied to Homepage (checks for PRODUCT STORE in body), Login (checks body does not contain Wrong password), and View Product (checks for Add to cart in body). This confirms the server returned meaningful correct content rather than an error page with a 200 status code.
- **Duration Assertion:** Applied to all five samplers with a threshold of 2,000 milliseconds. Fails the sample if the response time exceeds 2 seconds, directly

enforcing the response time SLA. A request can return HTTP 200 with correct content but still fail this assertion if it was too slow.

Listeners and Reporting

Listeners in JMeter collect and display test results. The following listeners are configured in each thread group.

- **Summary Report:** Displays an aggregated view of results for each sampler including sample count, average response time, minimum, maximum, error percentage, and throughput. This is the quickest way to review overall test results.
- **Aggregate Report:** Provides additional detail including response time percentiles at the 90th, 95th, and 99th percentile levels. These percentile values are more informative than averages because they reveal the experience of users at the slower end of the distribution.
- **View Results Tree:** Displays the full request and response details for each individual sample including headers, body, status code, and assertion results. Used for debugging failed requests during development. This listener is disabled during large-scale test runs to avoid excessive memory consumption.

III. Execution of Performance Tests

All test phases are executed in non-GUI command-line mode using JMeter's -n flag. Non-GUI mode is required for production test runs because the graphical interface consumes significant memory and CPU resources that would otherwise distort the load generation results. Tests are executed from the project root directory.

Each test phase produces a JTL results file, which is a CSV-formatted log containing one row per request sample with columns for timestamp, elapsed time, label, response code, success status, and other metrics. The JTL file is the raw output of a test run and serves as the input for generating the HTML Dashboard report.

The HTML Dashboard report is generated using JMeter's built-in reporting engine, either automatically during command-line execution using the -e and -o flags, or manually after the

test via the Tools menu in the JMeter GUI. The output directory must be empty before each report generation. Each test phase produces its own HTML Dashboard report stored in a dedicated subfolder.

IV. Reporting

HTML Dashboard reports are produced, one for each test phase: Baseline, Load Test, Peak Load, Stress Test, and Endurance Test. Each report contains interactive charts and statistics including response time over time, throughput over time, active threads over time, response time percentile distribution, error rate by transaction, and a complete statistics summary.

Reports are published automatically to GitHub Pages via the CI/CD pipeline after each test run. The landing page at the GitHub Pages URL provides an overview of all four phases with key metrics and links to each individual dashboard. A summary email is sent automatically after each pipeline run containing pass and fail counts, average response time, error rate, and a direct link to the live report.

Test Entry Criteria

The following conditions must all be met before performance testing can begin.

- Apache JMeter 5.6.3 must be installed and launchable. Java 21 must be confirmed as the active runtime by running java -version in a terminal.
- The DemoBlaze application at https://www.demoblaze.com must be accessible from the test machine. A manual smoke test involving login, add to cart, and place order must succeed without errors.
- The JMeter test plan file demoblaze_perf_test_fixed.jmx must have been validated with a small number of users (1 to 5) in the JMeter GUI to confirm all requests are correctly configured, all assertions pass, and the session token is extracted correctly from the login response.
- At least one valid DemoBlaze test account (username: perftest2026) must exist and be confirmed as functional by a manual login.

- All HTML report output directories must exist and be empty before each test run.
- The test machine must have a stable internet connection.

Test Exit Criteria

Testing is considered complete when all of the following conditions are met.

- All five test phases have been executed: Baseline (50 users), Load Test (150 users), Peak Load (500 users), Stress Test (1,000 users), and Endurance Test (50 users for 10 minutes).
- An HTML Dashboard report has been generated and verified for each of the five test phases.
- All test results have been reviewed against the performance goals and a pass or fail status has been assigned to each metric for each phase.
- The Final Performance Report summarising all findings, bottleneck analysis, and recommendations has been completed.
- The CI/CD pipeline has successfully completed at least one automated run with reports published to GitHub Pages.

Defect Management

This section defines the process for identifying, documenting, classifying, and resolving performance defects discovered during the DemoBlaze testing engagement. A structured defect management workflow ensures that performance bottlenecks and SLA breaches are communicated transparently and addressed systematically.

- **Defect Classification:** Performance defects are classified by severity based on measured impact. Critical defects are those where the error rate exceeds 5 percent or average response time exceeds 5,000 milliseconds under baseline conditions. High defects are SLA breaches where error rate exceeds 1 percent or average response time exceeds 2,000 milliseconds. Medium defects are P90 values exceeding 3,000

milliseconds or throughput falling below 500 requests per second at peak load. Low defects are minor response time increases that remain within SLA thresholds but show an upward trend across phases.

- **Defect Logging:** All defects identified during test execution are logged immediately with the test phase in which they were observed, the affected transaction name, the measured metric value, the SLA threshold breached, and an initial root cause hypothesis. Defects are tracked in the project repository and referenced in the Final Performance Report.
- **Root Cause Analysis:** The QA Engineer will investigate identified bottlenecks using JMeter HTML Dashboard reports, JTL results files, and response assertion failure messages to determine whether failures originate from server capacity limits, session management issues, database write contention, or network latency. Findings are documented against each logged defect.
- **Retesting:** Once a fix is implemented by the development team, the QA Engineer will re-execute the affected test phase under the same load conditions to verify that the defect is resolved and that no regression has been introduced in other transactions.
- **Defect Closure:** Defects will be closed only after the fix has been verified under the original load conditions, the error rate and response time return within defined SLA thresholds, and the result is documented in the Final Performance Report.

Testing Schedule

Phase	Timeline	Activities	Assigned To	Remarks
Planning & Execution	Before sprint	Define performance SLAs, workload model, and test scenarios with stakeholders	QA Engineers, Product Owner	SLA acceptance thresholds must be formally approved before scripting begins

Script Development & Data Prep	during sprint	Configure JMeter test plan with all five samplers, assertions, timers, and token extraction; prepare test account credentials	QA Engineers	Test plan to be validated manually with 1 to 5 users before full execution
Baseline & Load Testing	During sprint	Execute baseline test (50 users) and load tests (150 and 500 users); monitor response times, throughput, and error rates	QA Engineers	Capture baseline metrics for comparison with stress and endurance results
Stress Testing	During sprint	Gradually increase user load to 1,000 users beyond expected peak; identify system breaking points and performance degradation	QA Engineers	Document observed bottlenecks, error behaviour, and breaking point threshold
Endurance Testing	During sprint	Run sustained load test with 50 users continuously for 10 minutes; monitor for memory leaks and performance degradation over time	QA Engineers	Ensure system stability over extended operation with no upward trend in response times

Report Generation & CI/CD	End of sprint	Generate HTML Dashboard reports for all phases; publish to GitHub Pages via GitHub Actions; deliver Final Performance Report	QA Engineers, DevOps	All reports must be accessible via the GitHub Pages URL before sign-off

Risk and Mitigations

Shared Public Test Environment: DemoBlaze is a publicly hosted demo site with no uptime guarantee. Other testers may be running their own tests simultaneously, creating uncontrolled background load that inflates measured response times. Mitigation: execute tests during off-peak hours, run a manual smoke test before each phase, and record the exact time of each run.

Session Token Expiry During Endurance Test: If the authentication token expires during the 10-minute endurance run, all Add to Cart and Place Order requests will fail with 401 errors, inflating the error rate.

Mitigation: the login request is the first step in every loop iteration, refreshing the token at the start of each cycle.

Local Machine Resource Constraints: Running 500 or 1,000 concurrent users on a local machine can exhaust RAM or CPU, causing JMeter to become a bottleneck.

Mitigation: execute all high-concurrency tests in non-GUI command-line mode using the jmeter -n flag, and configure the JMeter heap to 2 gigabytes for peak and stress phases.

Network Latency Affecting Results: The test machine is located in Rwanda while the DemoBlaze servers are hosted on cloud infrastructure in North America or Europe. Network round-trip time adds 150 to 300 milliseconds to every measured response.

Mitigation: record and document baseline network latency before testing. All results are interpreted as relative comparisons between phases rather than absolute values.

Environment Setup

This section outlines the technical configuration of the performance test environment, including load generation infrastructure, application details, network conditions, and tools required to ensure accurate and reproducible performance test results.

Project Environments

The DemoBlaze performance testing engagement operates against a single publicly hosted environment. There is no separate development or staging environment available for this application.

- **Testing** : All performance test execution occurs against the live DemoBlaze public application. This is the only environment available for this engagement and is shared with other users and testers worldwide.
- **Production** : The DemoBlaze application at https://www.demoblaze.com is the live environment for end users and is identical to the testing target. All test traffic is directed to this environment.

All performance testing activities described in this plan are conducted against the public DemoBlaze environment. Because no isolated testing environment is available, tests are executed during off-peak hours to minimise interference from other concurrent users. Results are interpreted with this shared environment constraint in mind.

Test Environment

The load generation machine is the local workstation running Apache JMeter and is responsible for spawning virtual users, sending HTTP requests, collecting timing data, and writing results to disk.

- **Operating System** :Windows 11, 64-bit (AMD64 architecture)
- **JMeter Version** : Apache JMeter 5.6.3

- **Java Version** : Java 21 (Temurin LTS). JMeter 5.6.3 is fully compatible with Java 21. The active runtime is confirmed by running java -version in the terminal before each test session.
- **JMeter Heap Configuration** : Minimum 512 megabytes, maximum 1 gigabyte for baseline and load phases. Maximum 2 gigabytes for peak and stress phases to accommodate the higher thread count without out-of-memory errors.
- **CI/CD Environment** : GitHub Actions using the ubuntu-latest runner for all automated test executions. The runner provides 2 virtual CPUs and 7 gigabytes of RAM, which is sufficient for baseline and load phases. High-concurrency phases may require a self-hosted runner for more accurate results.

Test Data

The test plan uses a single registered DemoBlaze account with the username perftest2026. This account was manually created at https://www.demoblaze.com before testing began and validated by performing a manual login, add to cart, and place order sequence. The credentials are referenced in the 02 Login sampler request body. All Add to Cart requests generate a unique cart entry identifier at runtime using JMeter's built-in UUID function, ensuring each iteration creates a distinct cart entry rather than duplicating an existing one.

Devices and Browsers

Performance testing is conducted entirely via Apache JMeter running on Windows 11. No real browser is involved in test execution. JMeter sends raw HTTP requests directly at the network level, bypassing the browser rendering layer entirely. This means browser rendering time, JavaScript execution time, CSS loading, and image loading are not included in the measured response times. All response time metrics reflect pure server processing and network transmission time only, which is the standard approach for API-level performance testing.

Testing Tools

- **Apache JMeter 5.6.3** : The primary load testing tool used to simulate concurrent virtual users, execute HTTP requests, apply assertions, extract dynamic data via the Regular Expression Extractor, measure response times, and generate JTL results files and HTML Dashboard reports.

- **Java 21 (Temurin LTS)** : The Java runtime environment required by JMeter for execution.
- **GitHub Actions** : The CI/CD automation platform used to execute all test phases sequentially in a cloud environment, publish HTML Dashboard reports to GitHub Pages after each run, and send automated email notifications containing per-transaction pass and fail counts, average response time, error rate, and a direct link to the live report.
- **GitHub Pages** : Used to host and publicly serve the HTML Dashboard reports generated after each test run. Each phase report is accessible at a dedicated URL subfolder so all four phase reports are available simultaneously without overwriting each other.

Communications Plan

Daily Standup: A 15-minute meeting held every morning. The QA Engineer shares what tests were executed the previous day, what is planned for the current day, and any blockers such as the application being unavailable or a pipeline failure. The development team and technical lead attend to provide clarification on API behaviour and to receive early notice of performance findings.

Team Chat: A dedicated channel on Slack or Microsoft Teams is used for real-time communication throughout the testing period. Quick questions, configuration clarifications, and sharing of unexpected results are handled here.

Sprint Review: Held at the end of the sprint. The QA Engineer presents all test results, walks through the HTML Dashboard reports, highlights bottlenecks, and gathers feedback from stakeholders. The Product Owner uses this session to review findings against business requirements.

Risk Escalation: Any critical risk such as a confirmed breaking point or a sustained error rate above 5 percent is escalated immediately via direct message to the Product Owner and Technical Lead rather than being held until the next standup.

Automated CI/CD Notifications: An automated email is sent after every pipeline run containing pass and fail counts per transaction, average response time, error rate, P90 and P95 values, and a direct link to the live HTML Dashboard report on GitHub Pages.

Final Performance Report: A formal Word document produced once all test phases have been executed and results fully analysed. It contains data tables, comparative analysis across all phases, identified bottlenecks, root cause discussion, and prioritised recommendations. Distributed to the Product Owner and Instructor at the close of the engagement.